ECE 264 - Object-Oriented Software Development

Chapter 4

Functions

Core Objectives

- Create custom functions.
- Import and use Python Standard Library modules, such as `random` and `math`, to reuse code and avoid “reinventing the wheel.”
- Pass data between functions.
- Generate a range of random numbers.
- Learn simulation techniques using random-number generation.
- Seed the random number generator to ensure reproducibility.
- Pack values into a tuple and unpack values from a tuple.
- Return multiple values from a function via a tuple.
- Understand how an identifier’s scope determines where in your program you can use it
- Create functions with default parameter values.
- Call functions with keyword arguments.
- Create functions that can receive any number of arguments.
- Use methods of an object.

Why functions?

- Construct large programs from smaller, more manageable pieces.
- Divide and conquer.
- Use existing functions as building blocks.
  - Key aspect of software reusability.
  - Also a major benefit of object-oriented programming.
- Packaging code as a function allows you to execute it from various locations in your program just by calling the function.
- Makes programs easier to modify.

Define Functions

- Definition begins with the (**`def keyword`**, followed by the function name, a set of parentheses and a colon (:).
- By convention, function names should begin with a lowercase letter and in multiword names underscores should separate each word.
- Required parentheses contain the function's **parameter list**.
- Empty parentheses mean no parameters.
- The indented lines after the colon (:) are the function's **block**
  - Consists of an optional docstring followed by the statements that perform the function's task.

Function Call

- `square` function that calculates the square of its argument

```
def square(number):
    """Calculate the square of number."""
    return number ** 2
```

- Function calls:

```
square(7)
```

```
print('The square of 7 is', square(7))
```

- What happens when you call a function?
  - Parameters exist only during the function call.
  - Created on each call to the function to receive arguments.
  - Destroyed when the function returns its result to the caller.
  - A function's parameters and variables defined in its block are all **local variables**.
- `square?` and `square??`
  - Display its docstring
  - Displays the funstion's docstring and full source-code definition

Functions with Multiple Parameters

- `maximum` function that determines and returns the largest of three values.

```
def maximum(value1, value2, value3):
    """Return the maximum of three values."""
    max_value = value1
    if value2 > max_value:
        max_value = value2
    if value3 > max_value:
        max_value = value3
    return max_value
```

```
maximum(12, 27, 36)
```

```
36
```

```
maximum(12.3, 45.6, 9.7)
```

```
45.6
```

```
maximum('yellow', 'red', 'orange')
```

```
'yellow'
```

Random-Number Generation

- Python standard library's `random` module.

```
import random
```

```
for roll in range(10):
    print(random.randrange(1, 7), end=' ')
```

- `randrange` function generates an integer from the **first** argument value up to, but not including, the **second** argument value.
- Different values are displayed if you re-execute the loop.

Rolling a Die Example

- If `randrange` truly produces integers at random, every number in its range has an equal probability (or chance or likelihood) of being returned each time we call it.
- Roll a die 6,000,000 times.
- Each die face should occur approximately 1,000,000 times.
- We used Python's underscore (_) digit separator to make the value 6000000 more readable.

```python
# fig04_01.py
"""Roll a six-sided die 6,000,000 times."""
import random

# face frequency counters
frequency1 = 0
frequency2 = 0
frequency3 = 0
frequency4 = 0
frequency5 = 0
frequency6 = 0

# 6,000,000 die rolls
for roll in range(6_000_000):  # note underscore separators
    face = random.randrange(1, 7)

    # increment appropriate face counter
    if face == 1:
        frequency1 += 1
    elif face == 2:
        frequency2 += 1
    elif face == 3:
        frequency3 += 1
    elif face == 4:
        frequency4 += 1
    elif face == 5:
        frequency5 += 1
    elif face == 6:
        frequency6 += 1

print(f'Face{"Frequency":>13}')
print(f'{1:>4}{frequency1:>13}')
print(f'{2:>4}{frequency2:>13}')
print(f'{3:>4}{frequency3:>13}')
print(f'{4:>4}{frequency4:>13}')
print(f'{5:>4}{frequency5:>13}')
print(f'{6:>4}{frequency6:>13}')
```

```
Face    Frequency
   1      1000562
   2       999042
   3       999988
   4      1000966
   5       999281
   6      1000161
```

Reproducibility

- Function `randrange` generates pseudorandom numbers.
- Numbers appear to be random, because each time you start a new interactive session or execute a script that uses the random module's functions, Python internally uses a different seed value.
- When you are debugging logic errors in programs that use randomly generated data, it can be helpful to use the same sequence of random numbers.
- To do this, use the random module's `seed` function to seed the random-number generator:

```
random.seed(32)
```

```
for roll in range(10):
    print(random.randrange(1, 7), end=' ')
```

```
1 2 2 3 6 2 4 1 6 1
```

```
for roll in range(10):
    print(random.randrange(1, 7), end=' ')
```

```
1 3 5 3 1 5 6 4 3 5
```

```
random.seed(32)
```

```
for roll in range(10):
    print(random.randrange(1, 7), end=' ')
```

```
1 2 2 3 6 2 4 1 6 1
```

Case Study: A Game of Chance

- You roll two six-sided dice, each with faces containing one, two, three, four, five and six spots, respectively. When the dice come to rest, the sum of the spots on the two upward faces is calculated. If the sum is 7 or 11 on the first roll, you win. If the sum is 2, 3 or 12 on the first roll (called “craps”), you lose (i.e., the “house” wins). If the sum is 4, 5, 6, 8, 9 or 10 on the first roll, that sum becomes your “point.” To win, you must continue rolling the dice until you “make your point” (i.e., roll that same point value). You lose by rolling a 7 before making your point.

Python Standard Library

- You write Python programs by combining functions and classes (that is, custom types) that you create with preexisting functions and classes defined in modules, such as those in the Python Standard Library and other libraries.
- Avoid “reinventing the wheel.”
- A module is a file that groups related functions, data and classes.
- A **package** groups related modules.
- The Python Standard Library is provided with the core Python language.
- Its packages and modules contain capabilities for a wide variety of everyday programming tasks.

`math` Module Functions

- The **math module** defines functions for performing various common mathematical calculations.

Function	Description	Example
`ceil( x )`	Rounds x to the smallest integer not less than x	`ceil(9.2) is 10.0`, `ceil(-9.8) is -9.0`
`floor( x )`	Rounds x to the largest integer not greater than x	`floor(9.2) is 9.0`, `floor(-9.8) is -10.0`
`sin( x )`	Trigonometric sine of x (x in radians)	`sin(0.0) is 0.0`
`cos( x )`	Trigonometric cosine of x (x in radians)	`cos(0.0) is 1.0`
`tan( x )`	Trigonometric tangent of x (x in radians)	`tan(0.0) is 0.0`
`exp( x )`	Exponential function e^x	`exp(1.0) is 2.718282`, `exp(2.0) is 7.389056`
`log( x )`	Natural logarithm of x (base e)	`log(2.718282) is 1.0`, `log(7.389056) is 2.0`
`log10( x )`	Logarithm of x (base 10)	`log10(10.0) is 1.0`, `log10(100.0) is 2.0`
`pow( x, y )`	x raised to power y (x^y)	`pow(2.0, 7.0) is 128.0`, `pow(9.0, .5) is 3.0`
`sqrt( x )`	square root of x	`sqrt(900.0) is 30.0`, `sqrt(9.0) is 3.0`
`fabs( x )`	Absolute value of x—always returns a float. Python also has the built-in function `abs`, which returns an `int` or a `float`, based on its argument.	`fabs(5.1) is 5.1`, `fabs(-5.1) is 5.1`
`fmod( x, y )`	Remainder of x/y as a floating-point number	`fmod(9.8, 4.0) is 1.8`

Keyword Arguments

- When calling functions, you can use keyword arguments to pass arguments in any order.
- Each keyword *argument in a call* has the form *parametername=value*.
- Order of keyword arguments does not matter.

```
def rectangle_area(length, width):
    """Return a rectangle's area."""
    return length * width
```

```
rectangle_area(width=5, length=10)
```

Arbitrary Argument Lists

- Functions with **arbitrary argument lists**, such as built-in functions `min` and `max`, can receive *any* number of arguments.
- Function `min`'s documentation states that `min` has two *required* parameters (named `arg1` and `arg2`) and an optional third parameter of the form **`*args`**, indicating that the function can receive any number of additional arguments. The **`*args`** parameter must be the *rightmost* parameter.
- The **`*`** before the parameter name tells Python to pack any remaining arguments into a tuple that's passed to the **`args`** parameter.
- `average` function that can receive any number of arguments.

```
def average(*args):
    return sum(args) / len(args)
```

```
average(5, 10)
```

```
average(5, 10, 15)
```

```
average(5, 10, 15, 20)
```

Passing an Iterable's Individual Elements as Function Arguments

- Can unpack a tuple's, list's or other iterable's elements to pass them as individual function arguments.
- The *** operator**, when applied to an iterable argument in a function call, unpacks its elements.

```
grades = [88, 75, 96, 55, 83]
```

```
average(*grades)
```

Equivalent to `average(88, 75, 96, 55, 83)`.

Methods

- A method is simply a function that you call on an object using the form

```
object_name.method_name(arguments)
```

```
s = 'Hello'
```

```
s.lower()  # call lower method on string object s
```

```
s.upper()
```

```
s
```

Scope Rules

- Each identifier has a scope that determines where you can use it in your program.
- Complete scope details.
- Local Scope - A local variable's identifier has **local scope**.
- Global Scope - Identifiers defined outside any function (or class) have **global scope –** these may include functions, variables and classes.

Accessing a Global Variable from a Function

- By default, you cannot *modify* a global variable in a function
- Python creates a **new local variable** when you first assign a value to a variable in a function's block.
- In function `try_to_modify_global`'s block, the local x **shadows** the global x, making it inaccessible in the scope of the function's block.

```
x = 7
```

```
def access_global():
    print('x printed from access_global:', x)
```

```
access_global()
```

```
def try_to_modify_global():
    x = 3.5
    print('x printed from try_to_modify_global:', x)
```

```
try_to_modify_global()
```

Accessing a Global Variable from a Function

- To modify a global variable in a function's block, you must use a **global** statement to declare that the variable is defined in the global scope:

```
def modify_global():
    global x;
    x = 'hello'
    print('x printed from modify_global:', x)
```

```
modify_global()
```

Blocks vs. Suites

- When you create a variable in a block, it's *local* to that block.
- When you create a variable in a control statement's suite, the variable's scope depends on where the control statement is defined:
  - If it's in the global scope, any variables defined in the control statement have **global scope**.
  - If it's in a function's block, any variables defined in the control statement have **local scope**.
- Script statements at global scope execute as soon as they are encountered by the interpreter, whereas statements in a block execute only when the function is called.

`import`

- **Importing Multiple Identifiers from a Module**
- Use `from…import` to import a comma-separated list of identifiers from a module then use them without having to precede them with the module name and a dot (.):

```
from math import ceil, floor
```

```
ceil(10.3)
```

```
floor(10.7)
```

- **Binding Names for Modules and Module Identifiers**

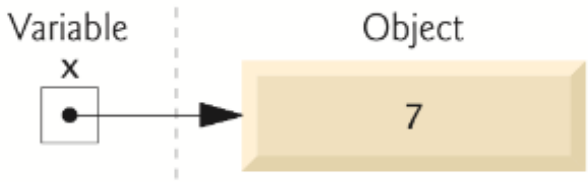
```
import statistics as stats
```

```
grades = [85, 93, 45, 87, 93]
```

```
stats.mean(grades)
```

Passing Arguments to Functions

- **Python arguments are always passed by reference**.
- Some people call this **pass-by-object-reference**, because "everything in Python is an object."
- When a function call provides an argument, Python copies the argument object's *reference*—not the object itself—into the corresponding parameter.
- Variable x contains a reference to an object containing 7 stored elsewhere in memory.



Passing an Object to a Function

- The identity displayed for cube's parameter number is the *same* as that displayed for x previously.
- The *argument* x and the *parameter* number refer to the *same object* while cube executes.

```
x = 7
```

```
id(x)
```

```
4307493256
```

```
def cube(number):
    print('id(number):', id(number))
    return number ** 3
```

```
cube(x)
```

```
id(number): 4307493256
343
```

Immutable Objects as Arguments

- When a function receives as an argument a reference to an *immutable* (unmodifiable) object, even though you have direct access to the original object in the caller, you cannot modify the original immutable object's value.

```
def cube(number):
    print('id(number) before modifying number:', id(number))
    number **= 3
    print('id(number) after modifying number:', id(number))
    return number
```

```
cube(x)
```

```
id(number) before modifying number: 4369408392
id(number) after modifying number: 4427551408
343
```

```
print(f'x = {x}; id(x) = {id(x)}')
```

```
x = 7; id(x) = 4369408392
```